

Chapters 10-13 Summaries — Elementary Data Structures Through Red-Black Trees

Ch. 10 — Elementary Data Structures

Named Entities (Terms & Definitions)

- Array: contiguous sequence of bytes; element i at address $a + b(i - s)$ for 1-origin ($s=1$) or $a + bi$ for 0-origin ($s=0$); $O(1)$ access
- Matrix: $m \times n$ grid; stored row-major or column-major in one or more arrays
- Row-major order: $M[i,j]$ at index $n(i-1)+j$ (1-origin); rows stored contiguously
- Column-major order: $M[i,j]$ at index $i + m(j-1)$ (1-origin); columns stored contiguously
- Block representation: matrix divided into blocks, each stored contiguously
- Ragged array: multiple-array representation where rows may have different lengths
- Stack: LIFO dynamic set; INSERT=PUSH, DELETE=POP; attribute S.top
- Queue: FIFO dynamic set; INSERT=ENQUEUE, DELETE=DEQUEUE; attributes Q.head, Q.tail; uses circular array
- Dequeue: double-ended queue; insertion/deletion at both ends
- Linked list: linear order determined by pointers (not array indices)
- Doubly linked list: each node has key, next, prev pointers
- Singly linked list: each node has next but no prev pointer
- Sorted list: linear order corresponds to sorted key order
- Circular list: tail.next = head, head.prev = tail
- Sentinel: dummy node (L.nil) simplifying boundary conditions; replaces NIL references; creates circular doubly linked list with sentinel
- Left-child, right-sibling representation: each node has x.left-child (leftmost child) and x.right-sibling (next sibling); $O(n)$ space for n -node tree
- Trailing pointer: pointer y maintained as parent of x during tree traversal (used in TREE-INSERT)
- Compact list: singly linked list stored in two arrays $key[1:n]$ and $next[1:n]$ with head pointer

Processes / Algorithms / Pathways

STACK-EMPTY(S)

- Steps: (1) if S.top == 0 return TRUE (2) else return FALSE
- Complexity: $O(1)$

PUSH(S, x)

- Goal: Insert element x onto stack
- Steps: (1) if S.top == S.size $\rightarrow$ error “overflow” (2) S.top = S.top + 1 (3) S[S.top] = x

- Complexity: $O(1)$
- Example: Stack $S = [1,4,9,16]$, $S.top=4$. $PUSH(S,25)$: $S.top=5$, $S[5]=25$. Stack now $[1,4,9,16,25]$.

POP(S)

- Goal: Remove and return top element
- Steps: (1) if $STACK-EMPTY(S) \rightarrow$ error "underflow" (2) $S.top = S.top - 1$ (3) return $S[S.top + 1]$
- Complexity: $O(1)$

ENQUEUE(Q, x)

- Steps: (1) $Q[Q.tail] = x$ (2) if $Q.tail == Q.size \rightarrow Q.tail = 1$ else $Q.tail = Q.tail + 1$
- Complexity: $O(1)$
- Circular wrap: location 1 follows location n ; when $Q.head == Q.tail$ the queue is empty

DEQUEUE(Q)

- Steps: (1) $x = Q[Q.head]$ (2) if $Q.head == Q.size \rightarrow Q.head = 1$ else $Q.head = Q.head + 1$ (3) return x
- Complexity: $O(1)$

LIST-SEARCH(L, k)

- Steps: (1) $x = L.head$ (2) while $x \neq NIL$ and $x.key \neq k$: $x = x.next$ (3) return x
- Complexity: $\Theta(n)$ worst-case

LIST-PREPEND(L, x)

- Steps: (1) $x.next = L.head$ (2) $x.prev = NIL$ (3) if $L.head \neq NIL$: $L.head.prev = x$ (4) $L.head = x$
- Complexity: $O(1)$

LIST-INSERT(x, y)

- Steps: Insert x immediately after y : (1) $x.next = y.next$ (2) $x.prev = y$ (3) if $y.next \neq NIL$: $y.next.prev = x$ (4) $y.next = x$
- Complexity: $O(1)$; does not reference list object L

LIST-DELETE(L, x)

- Steps: (1) if $x.prev \neq NIL$: $x.prev.next = x.next$ else $L.head = x.next$ (2) if $x.next \neq NIL$: $x.next.prev = x.prev$
- Complexity: $O(1)$ (plus $\Theta(n)$ to find element via LIST-SEARCH)

LIST-DELETE'(x) — sentinel version

- Steps: (1) $x.\text{prev}.\text{next} = x.\text{next}$ (2) $x.\text{next}.\text{prev} = x.\text{prev}$
- Complexity: $O(1)$; no boundary checks needed since sentinel $L.\text{nil}$ is always present

LIST-INSERT'(x, y) — sentinel version

- Steps: (1) $x.\text{next} = y.\text{next}$ (2) $x.\text{prev} = y$ (3) $y.\text{next}.\text{prev} = x$ (4) $y.\text{next} = x$
- Complexity: $O(1)$; to prepend set $y = L.\text{nil}$; to append set $y = L.\text{nil}.\text{prev}$

LIST-SEARCH'(L, k) — sentinel optimizes constant factor

- Steps: (1) $L.\text{nil}.\text{key} = k$ (2) $x = L.\text{nil}.\text{next}$ (3) while $x.\text{key} \neq k$: $x = x.\text{next}$ (4) if $x == L.\text{nil}$ return NIL else return x
- Complexity: $\Theta(n)$; eliminates one comparison per iteration (no NIL-check on each node)

Comparisons & Trade-offs

Dimension	Array	Doubly Linked List
Access k-th element	$O(1)$	$\Theta(k)$
Insert at front	$\Theta(n)$ (must shift)	$O(1)$
Delete first element	$\Theta(n)$	$O(1)$
Search	$\Theta(n)$ (unsorted)	$\Theta(n)$
Space	n elements	n elements + $2n$ pointers
Sentinels benefit	N/A	Simplifies code, small constant speedup

Formulas & Equations

Array element address (1-origin) $\text{address} = a + b(i - 1)$ where a = base address, b = element size

Row-major index (1-origin) $\text{index} = n(i - 1) + j$ where n = columns, i = row, j = column

Column-major index (1-origin) $\text{index} = i + m(j - 1)$ where m = rows

Row-major index (0-origin) $\text{index} = ni + j$

Column-major index (0-origin) $\text{index} = i + mj$

Edge Cases & Common Pitfalls

- Stack underflow: POP from empty stack ($S.top = 0$)
- Stack overflow: PUSH to full stack ($S.top = S.size$)
- Queue underflow: DEQUEUE from empty queue ($Q.head = Q.tail$)
- Queue overflow: ENQUEUE to full queue ($Q.head = Q.tail + 1$ in circular sense); max $n-1$ elements in n -size array
- Singly linked list deletion: $O(n)$ worst-case because predecessor must be found
- Sentinel overuse: wastes memory on many small lists
- Deletion with sentinel: never delete the sentinel itself
- Sentinel key: LIST-SEARCH' temporarily stores search key in sentinel — must not be NIL if NIL is used for empty-key detection

Diagrams & Visuals

Array implementation of stack $S[1:n]$:

```
bottom → [e1] [e2] [e3] [e4] [ ] ... [ ]
           ↑ S.top
```

Circular queue $Q[1:n]$:

```
head → [ ] [ ] [ ] [15] [6] [9] [8] [4] ← tail
        ↑           ↑
      tail=1      head=4
```

Doubly linked list with sentinel:

```
L.nil  head-node  ...  tail-node  L.nil
```

Binary tree representation:

```
T.root → [p, left, right]
          /      \
        [p, l, r]  [p, l, r]
```

Left-child, right-sibling:

```
parent → left-child → right-sibling → right-sibling → NIL
```

End-of-Chapter Material

- Key terms: array, matrix, row-major, column-major, block representation, stack (LIFO), queue (FIFO), deque, linked list, doubly/singly linked, sorted list, circular list, sentinel, left-child right-sibling representation, trailing pointer, compact list
- Exercises 10.1: implement two stacks in one array; implement queue using two stacks; implement stack using two queues
- Exercises 10.2: implement stack/queue with singly linked list; reverse singly linked list in $\Theta(n)$ with constant space; XOR doubly linked list ($x.next = x.next XOR x.prev$)
- Problems 10-1: comparison of list types (unsorted/sorted, singly/doubly) for SEARCH, INSERT, DELETE, SUCCESSOR, PREDECESSOR, MINIMUM, MAX-

IMUM

- Problems 10-2: mergeable heaps using linked lists — sorted, unsorted, and unsorted with disjoint merge
- Problems 10-3: COMPACT-LIST-SEARCH — randomized algorithm that skips ahead to random position; expected $O(\sqrt{n})$ time; analysis via COMPACT-LIST-SEARCH' with parameter t , showing $E[Xt] \leq n/(t+1)$

Cross-Chapter Links

- Requires: Ch. 2 (RAM model, Big-O notation), Appendix B (tree math)
 - Referenced in: Ch. 6 (heaps as arrays), Ch. 11 (chaining uses linked lists), Ch. 12 (BST uses binary tree representation, trailing pointer), Ch. 13 (red-black trees use sentinel), Ch. 18 (B-trees), Ch. 19 (disjoint-set forests)
-

Ch. 11 — Hash Tables

Named Entities (Terms & Definitions)

- Direct-address table: array $T[0:m-1]$ where slot k stores element with key k ; $O(1)$ dictionary operations; requires universe U reasonably small
- Hash table: array $T[0:m-1]$ where element with key k stored in slot $h(k)$; size $m \ll |U|$
- Hash function $h: U \rightarrow \{0, 1, \dots, m-1\}$; maps keys to slots
- Collision: two distinct keys hash to same slot
- Load factor α : $\alpha = n/m$ (n elements, m slots); for chaining α can be > 1 ; for open addressing $\alpha \leq 1$
- Chaining: each slot points to linked list of keys hashing to that slot
- Independent uniform hashing: $h(k)$ uniformly random from $\{0, \dots, m-1\}$, independent across keys
- Universal hashing: family H where for any distinct k, k' , $\Pr[h(k) = h(k')] \leq 1/m$
- ϵ -universal: collision probability $\leq \epsilon$
- d -independent: for any d distinct keys, $\Pr[h(k) = q \ \forall i] = 1/m$
- Division method: $h(k) = k \bmod m$ (m prime, not near power of 2 avoids patterns in low-order bits)
- Multiplication method: $h(k) = \lfloor m(kA \bmod 1) \rfloor$, $0 < A < 1$; common choice $A = (\sqrt{5}-1)/2 \approx 0.6180339$
- Multiply-shift: $h(k) = (ka \bmod 2^w) \ggg (w-\ell)$; for $m=2^\ell$, w -bit words; $2/m$ -universal
- Open addressing: all elements in table; no external storage; $\alpha \leq 1$
- Probe sequence: $\langle h(k,0), h(k,1), \dots, h(k,m-1) \rangle$ — must be permutation of $\{0, \dots, m-1\}$
- Double hashing: $h(k,i) = (h(k) + i \cdot h'(k)) \bmod m$; requires $h(k)$ relatively prime to m ; produces $\Theta(m)$ distinct probe sequences
- Linear probing: $h(k,i) = (h(k) + i) \bmod m$; suffers primary clustering; excellent cache locality
- Independent uniform permutation hashing: probe sequence of each key equally likely any permutation; idealized model

- Primary clustering: long runs of occupied slots in linear probing; empty slot preceded by i full slots filled with probability $(i+1)/m$
- Secondary clustering: two keys with same $h(k)$ follow same probe sequence in double hashing (but different from linear probing's primary clustering)
- Cryptographic hash function: SHA-256 produces 256-bit output; usable via $h(k) = \text{SHA-256}(k) \bmod m$
- Wee hash function: $f(k) = \text{swap}((2k + ak) \bmod 2^r)$; iterate r rounds with CBC-style chaining for variable-length inputs; inspired by RC6
- Perfect hashing: two-level scheme with no collisions; first-level hash maps to buckets, second-level collision-free per-bucket hash; $O(n)$ expected construction time
- Random oracle: idealization where $h(k)$ is truly random function; used for theoretical analysis
- Symbol table: application where hash tables store identifiers and their attributes

Processes / Algorithms / Pathways

DIRECT-ADDRESS-SEARCH(T, k)

- Steps: return $T[k]$
- Complexity: $O(1)$

CHAINED-HASH-INSERT(T, x)

- Steps: LIST-PREPEND($T[h(x.key)], x$)
- Complexity: $O(1)$ worst-case
- Example: T size 9, $h(k)=k \bmod 9$. Insert 5,28,19,15,20,33,12,17,10. $h(5)=5$, $h(28)=1$, $h(19)=1$ (chain: $19 \rightarrow 28$), $h(15)=6$, $h(20)=2$, $h(33)=6$ (chain: $33 \rightarrow 15$), $h(12)=3$, $h(17)=8$, $h(10)=1$ (chain: $10 \rightarrow 19 \rightarrow 28$)

CHAINED-HASH-SEARCH(T, k)

- Steps: return LIST-SEARCH($T[h(k)], k$)
- Complexity: $\Theta(1+\alpha)$ average under independent uniform hashing (Theorem 11.1 unsuccessful, Theorem 11.2 successful); $\Theta(n)$ worst-case

CHAINED-HASH-DELETE(T, x)

- Steps: LIST-DELETE($T[h(x.key)], x$)
- Complexity: $O(1)$ if doubly linked list

HASH-INSERT(T, k) — open addressing

- Steps: (1) $i=0$ (2) repeat: $q = h(k,i)$; if $T[q] == \text{NIL}$: $T[q]=k$, return q ; else $i=i+1$ (3) until $i==m$ (4) error "hash table overflow"
- Complexity: Expected $O(1/(1-\alpha))$ probes under independent uniform permutation hashing

HASH-SEARCH(T, k) — open addressing

- Steps: (1) $i=0$ (2) repeat: $q = h(k,i)$; if $T[q]==k$ return q ; $i=i+1$ (3) until $T[q]==NIL$ or $i==m$ (4) return NIL

LINEAR-PROBING-HASH-DELETE(T, q)

- Steps: (1) $T[q]=NIL$ (2) search forward; for each key k' found, compute $g(k',q) = (q - h(k')) \bmod m$; if $g(k',q) < g(k',q')$, move k' to vacated slot; continue until empty slot found
- Complexity: proportional to distance from $h(k)$ to next empty slot; avoids needing DELETED marker

Classifications & Hierarchies

- Hash function families by strength:
 - Uniform: $\Pr[h(k)=q] = 1/m$ for any key k
 - Universal: $\Pr[h(k)=h(k')] \leq 1/m$ for distinct keys
 - ϵ -universal: collision prob $\leq \epsilon$
 - d -independent: d -wise joint distribution uniform
- Collision resolution:
 - Chaining (uses external lists; α can exceed 1)
 - Open addressing (all in table; $\alpha \leq 1$): linear probing, double hashing
- Hash function construction:
 - Static: division; multiplication; multiply-shift (fixed h)
 - Random: universal family $H = \{h_b(k) = ((ak+b) \bmod p) \bmod m\}$; multiply-shift with random odd a
 - Cryptographic: SHA-256 + salt; wee hash (RC6-based)
- Perfect hashing: two-level scheme; first level maps keys to buckets; second level uses collision-free per-bucket hash; static set, $O(1)$ worst-case search

Comparisons & Trade-offs

Dimension	Chaining	Open Addressing
Load factor α	Can exceed 1	≤ 1
Storage	table + pointers	table only (no pointers)
Deletion	Trivial ($O(1)$ doubly linked)	Tricky; needs DELETED marker or linear-probing delete
Search avg ($\alpha=.5$)	$\Theta(1+\alpha) \approx 1.5$ probes	$1/(1-\alpha) \approx 2$ probes (unsuccessful)
Cache behavior	Poor (list traversal)	Better (linear probing sequential)
Worst-case	$\Theta(n)$ (all same slot)	$\Theta(n)$ (table full, must probe all)

Dimension	Linear Probing	Double Hashing
Probe sequences	m distinct	$\Theta(m)$ distinct
Primary clustering	Yes	No
Secondary clustering	N/A	Two keys with same h have same probe sequence
Cache locality	Excellent	Poor
Deletion w/o DELETED	Yes (L-P delete)	No

Dimension	Division Method	Multiply-Shift	Universal Family H
Speed	Very fast	3 instructions	Moderate (needs mod p)
m constraint	Prime preferred, not near 2	Power of 2 ($m=2^\ell$)	Arbitrary
Guarantee	None (static)	None (static)	Universal ($\leq 1/m$ collision)

Formulas & Equations

Load factor $\alpha = n / m$

Expected unsuccessful search — chaining (Theorem 11.1) $\Theta(1 + \alpha)$ under independent uniform hashing

Expected successful search — chaining (Theorem 11.2) $\Theta(1 + \alpha)$ under independent uniform hashing

Expected probes unsuccessful search — open addressing (Theorem 11.6) $1 / (1 - \alpha)$ under independent uniform permutation hashing, $\alpha < 1$

Expected probes successful search — open addressing (Theorem 11.8) $(1/\alpha) \ln(1/(1-\alpha))$ under independent uniform permutation hashing

Universal hash family H (Theorem 11.4) $h_b(k) = ((ak + b) \bmod p) \bmod m$ - $a \in \{1, \dots, p-1\}$, $b \in \{0, \dots, p-1\}$, p prime $> m$ - $|H| = p(p-1)$

Multiply-shift hash (Theorem 11.5) $h(k) = (ka \bmod 2^w) \bmod m$ - $(w-1) - m = 2^\ell$, w-bit words, a odd - 2/m-universal

Double hashing $h(k, i) = (h(k) + i \cdot h'(k)) \bmod m$ - $h(k)$ must be relatively prime to m - Common: m prime, $h'(k) = 1 + (k \bmod m')$, m' slightly $< m$

Rules, Laws & Theorems

Theorem 11.1

- Chaining unsuccessful search expected time $\Theta(1+\alpha)$ under independent uniform hashing

Theorem 11.2

- Chaining successful search expected time $\Theta(1+\alpha)$ under independent uniform hashing

Theorem 11.4 (H universal)

- $H = \{h_b: h_b(k) = ((ak+b) \bmod p) \bmod m\}$ for prime $p > m$ is universal

Theorem 11.5 (multiply-shift universal)

- $h(k) = (ka \bmod 2^w) \ll w$ with odd a is $2^w/m$ -universal

Theorem 11.6 (open addressing unsuccessful)

- Under independent uniform permutation hashing, expected probes $\leq 1/(1-\alpha)$

Theorem 11.8 (open addressing successful)

- Under independent uniform permutation hashing, expected probes $\leq (1/\alpha) \ln(1/(1-\alpha))$

Theorem 11.9 (linear probing with 5-independence)

- With 5-independent h and $\alpha \leq 2/3$, expected operations take constant time

Edge Cases & Common Pitfalls

- Worst-case chaining: all keys hash to same slot $\rightarrow \Theta(n)$ per operation
- m power of 2 with division: $h(k) = k \bmod 2^\ell$ uses only low-order ℓ bits — bad if keys are patterned
- Open addressing deletion: setting slot to NIL breaks search; use DELETED marker instead (but then α no longer bounds search time) or linear-probing delete
- Double hashing non-relatively-prime: if $\gcd(h(k), m) = d > 1$, only $1/d$ of table probed
- Linear probing primary clustering: empty slot preceded by i full slots filled with probability $(i+1)/m$
- $\alpha \rightarrow 1$ in open addressing: probes $\rightarrow \infty$; unsuccessful search at $\alpha=1$ probes all m slots
- Static hash function: adversary can choose n keys all same slot
- Multiplication method choice of A : Knuth suggests $A \approx (\sqrt{5}-1)/2 \approx 0.6180339887$
- 5-independence needed: linear probing with only 2-independent hash may have non-constant expected time

Diagrams & Visuals

Direct-address table:

Keys $U=\{0,\dots,9\}$ $K=\{2,3,5,8\}$

T: [NIL] [NIL] [→e] [→e] [NIL] [→e] [NIL] [NIL] [→e] [NIL]
0 1 2 3 4 5 6 7 8 9

Chaining:

T: [NIL] → [→k →k] → [→k →k →k] → [NIL] → ...
0 1 2 3

Multiply-shift:

k (w bits) $\rightarrow \times a$ (w bits) $\rightarrow [r \text{ (high } w) \mid r \text{ (low } w)] \rightarrow \text{take MSB of } r$

End-of-Chapter Material

- Key terms: direct-address table, hash table, hash function, collision, chaining, load factor, independent uniform hashing, universal hashing, ϵ -universal, d -independent, division method, multiplication method, multiply-shift, open addressing, probe sequence, double hashing, linear probing, primary/secondary clustering, cryptographic hash, wee hash, random oracle, perfect hashing, symbol table
- Theorem 11.1–11.2: chaining search $\Theta(1+\alpha)$ average
- Theorem 11.4: H is universal
- Theorem 11.5: multiply-shift is $2/m$ -universal
- Theorem 11.6: open addressing unsuccessful search $\leq 1/(1-\alpha)$
- Theorem 11.8: open addressing successful search $\leq (1/\alpha) \ln(1/(1-\alpha))$
- Theorem 11.9: linear probing with 5-independent h and $\alpha \leq 2/3$ takes expected constant time
- Exercises 11.1: direct-address table with multiple objects same key; bitmap to track empty slots; satellite data in table
- Exercises 11.2: chaining slot-size bound; expectation of chain length squared; hat-check problem; randomized strategy for checking equality
- Exercises 11.3: division method with $m=2^p$; multiplication method examples; H universal proof; string keys as base-128 numbers
- Exercises 11.4: open addressing double hashing; probe sequence permutations; primary clustering analysis
- Problem 11-1: longest-probe bound for chaining; show max chain length $M = O(\lg n / \lg \lg n)$ w.h.p.
- Problem 11-2: slot-size bound for chaining
- Problem 11-3: perfect hashing — two-level scheme with $O(n)$ construction, $O(1)$ worst-case search

Cross-Chapter Links

- Requires: Ch. 10 (linked lists for chaining, sentinels), Ch. 5 (indicator random variables, expectation, tail bounds), Ch. 31 (modular arithmetic for universal hashing)

- Referenced in: Ch. 16 (amortized analysis of dynamic tables), Ch. 22 (graph representations via hash tables), Ch. 32 (Rabin-Karp string matching uses rolling hash)
-

Ch. 12 — Binary Search Trees

Named Entities (Terms & Definitions)

- Binary search tree (BST): binary tree with BST property: for any node x , all keys in left subtree $\leq x.key \leq$ all keys in right subtree
- BST property: if y in left subtree of x then $y.key \leq x.key$; if y in right subtree then $y.key \geq x.key$
- Inorder tree walk: recursively visit left, print root, visit right; prints keys in sorted order in $\Theta(n)$ time
- Preorder tree walk: print root, visit left, visit right
- Postorder tree walk: visit left, visit right, print root
- Successor: next node visited in inorder walk; if right subtree exists $\rightarrow$ minimum of right subtree; else lowest ancestor whose left child is also ancestor
- Predecessor: symmetric to successor; last node visited before x in inorder walk
- Trailing pointer: pointer y maintained as parent of x during tree descent (used in TREE-INSERT)
- TRANSPLANT: subroutine replacing one subtree with another; does not update children of v
- Radix tree (trie): tree on bit strings; go left if bit=0, right if bit=1; sorts strings in $\Theta(\text{total length})$ time
- Randomly built BST: insert n keys in random order (all $n!$ permutations equally likely); expected height $O(\lg n)$
- Catalan number: $b_n = (1/(n+1))C(2n,n)$; number of distinct binary trees with n nodes
- Total path length: sum over all nodes of depth; $P(T) = \sum_{x \in T} d(x,T)$
- BST sort: insert n keys then inorder walk; worst-case $\Theta(n^2)$, best-case $\Theta(n \lg n)$

Processes / Algorithms / Pathways

INORDER-TREE-WALK(x)

- Goal: Print keys of BST in sorted order
- Steps: (1) if $x \neq \text{NIL}$: (2) INORDER-TREE-WALK($x.\text{left}$) (3) print $x.\text{key}$ (4) INORDER-TREE-WALK($x.\text{right}$)
- Complexity: $\Theta(n)$ (Theorem 12.1: proof by substitution, $T(n) \leq (c+d)n + c$)
- Example: Tree root=6, left=5 (with left=2, right=5), right=7 (with right=8).
Output: 2,5,5,6,7,8

TREE-SEARCH(x, k)

- Goal: Find node with key k in subtree rooted at x

- Steps: (1) if $x == \text{NIL}$ or $k == x.\text{key}$: return x (2) if $k < x.\text{key}$: return $\text{TREE-SEARCH}(x.\text{left}, k)$ (3) else: return $\text{TREE-SEARCH}(x.\text{right}, k)$
- Complexity: $O(h)$ where h = tree height
- Path: forms simple path downward from root; $O(h)$ time

ITERATIVE-TREE-SEARCH(x, k)

- Steps: (1) while $x \neq \text{NIL}$ and $k \neq x.\text{key}$: (2) if $k < x.\text{key}$: $x = x.\text{left}$ (3) else: $x = x.\text{right}$ (4) return x
- Complexity: Same, more efficient in practice (unrolls recursion)

TREE-MINIMUM(x)

- Goal: Find minimum key in subtree
- Steps: (1) while $x.\text{left} \neq \text{NIL}$: $x = x.\text{left}$ (2) return x
- Complexity: $O(h)$

TREE-MAXIMUM(x)

- Steps: (1) while $x.\text{right} \neq \text{NIL}$: $x = x.\text{right}$ (2) return x
- Complexity: $O(h)$

TREE-SUCCESSOR(x)

- Goal: Find next node in inorder traversal
- Steps: (1) if $x.\text{right} \neq \text{NIL}$: return $\text{TREE-MINIMUM}(x.\text{right})$ (2) $y = x.p$ (3) while $y \neq \text{NIL}$ and $x == y.\text{right}$: $x = y$; $y = y.p$ (4) return y
- Complexity: $O(h)$
- Two cases: (a) right subtree nonempty $\rightarrow$ leftmost in right subtree; (b) right subtree empty $\rightarrow$ lowest ancestor whose left child is also ancestor

TREE-PREDECESSOR(x)

- Steps: symmetric to SUCCESSOR ; if $x.\text{left} \neq \text{NIL}$: return $\text{TREE-MAXIMUM}(x.\text{left})$; else go up until x is right child

TREE-INSERT(T, z)

- Goal: Insert node z into BST T ($z.\text{key}$ set, children NIL)
- Steps: (1) $x = T.\text{root}$; $y = \text{NIL}$ (2) while $x \neq \text{NIL}$: $y = x$; if $z.\text{key} < x.\text{key}$: $x = x.\text{left}$ else $x = x.\text{right}$ (3) $z.p = y$ (4) if $y == \text{NIL}$: $T.\text{root} = z$ (5) elseif $z.\text{key} < y.\text{key}$: $y.\text{left} = z$ (6) else: $y.\text{right} = z$
- Complexity: $O(h)$
- Trailing pointer: y trails behind x ; when x hits NIL , y is the parent for z
- Example: Insert key 13 into tree $\text{root}=15$, $\text{left}=6$, $\text{right}=20$. Path: $15 \rightarrow 6 \rightarrow 7 \rightarrow \text{NIL}$ (right of 7). Insert as right child of 7.

TRANSPLANT(T, u, v)

- Goal: Replace subtree rooted at u with subtree rooted at v
- Steps: (1) if u.p == NIL: T.root = v (2) elseif u == u.p.left: u.p.left = v (3) else: u.p.right = v (4) if v ≠ NIL: v.p = u.p
- Note: does not update v.left or v.right — caller's responsibility

TREE-DELETE(T, z)

- Goal: Delete node z from BST T
- Cases:
 - (a) z no left child → TRANSPLANT(z, z.right)
 - (b) z has left but no right child → TRANSPLANT(z, z.left)
 - (c) z two children, successor y is z.right → TRANSPLANT(z, y); y.left = z.left
 - (d) z two children, successor y deeper → TRANSPLANT(y, y.right); y.right = z.right; TRANSPLANT(z, y); y.left = z.left
- Complexity: O(h)
- Example: Delete root 15 (has children 6 and 20). y = TREE-MINIMUM(z.right) = 17. If 17 not right child: splice 17 out, replace with 17's right child, set 17.right = 20, TRANSPLANT(15, 17), set 17.left = 6.

Classifications & Hierarchies

- BST variants:
 - Basic BST (Ch. 12): O(h) operations, h could be n
 - Red-black tree (Ch. 13): $h \leq 2 \lg(n+1)$
 - AVL tree (Problem 13-3): $|\text{left.h} - \text{right.h}| \leq 1$; $h \leq 1.44 \lg n$
 - Radix tree/trie (Problem 12-2): bit-based keys
 - Optimal BST (Ch. 14): known search frequencies
- Tree walks: inorder (sorted), preorder (root first), postorder (root last)

Comparisons & Trade-offs

Dimension	Unsorted Array	Sorted Array	BST (height h)	Red-Black Tree
SEARCH	$\Theta(n)$	$O(\log n)$	$O(h)$	$O(\log n)$
INSERT	$O(1)$	$\Theta(n)$ (shift)	$O(h)$	$O(\log n)$
DELETE	$\Theta(n)$ (search+shift)	$\Theta(n)$	$O(h)$	$O(\log n)$
MIN/MAX	$\Theta(n)$	$O(1)$	$O(h)$	$O(\log n)$
SUCCESSOR	N/A	$O(1)$ if index given	$O(h)$	$O(\log n)$
Sorted output	$\Theta(n \log n)$ sort	$\Theta(n)$ walk	$\Theta(n)$ inorder	$\Theta(n)$ inorder

Formulas & Equations

Inorder walk recurrence $T(0) = c$; $T(n) = T(k) + T(n-k-1) + d \rightarrow T(n) = (c+d)n + c$
 $\rightarrow T(n) = \Theta(n)$

BST height bound (random) $E[\text{height}] = O(\log n)$ for random insertion order

Number of distinct binary trees (Catalan) $b = (1/(n+1)) \cdot C(2n, n) = \Omega(4^n / n^{3/2})$ - Recurrence: $b = 1$, $b = \sum_{k=0}^{n-1} b \cdot b_{n-1-k}$

BST sort (insert + inorder) Worst-case: $\Theta(n)$ (sorted/almost-sorted input); Best-case: $\Theta(n \lg n)$ (balanced)

Rules, Laws & Theorems

Theorem 12.1

- Statement: INORDER-TREE-WALK on n -node subtree takes $\Theta(n)$ time
- Proof: $T(n) \leq T(k) + T(n-k-1) + d$; substitution $T(n) \leq (c+d)n + c$

Theorem 12.2

- Statement: SEARCH, MINIMUM, MAXIMUM, SUCCESSOR, PREDECESSOR each run in $O(h)$ on BST of height h

Theorem 12.3

- Statement: INSERT and DELETE run in $O(h)$ on BST of height h

Lower bound for BST construction

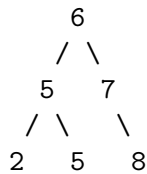
- Any comparison-based BST construction from n elements takes $\Omega(n \lg n)$ worst-case (sorting lower bound)

Edge Cases & Common Pitfalls

- Degenerate BST: inserting sorted keys yields $h = n$ (linked list); operations $\Theta(n)$
- Equal keys: need convention about $\leq$ vs $<$ in BST property; Problem 12-1 explores strategies (boolean flag, list at node, randomization)
- Deleting node with two children: two subcases — successor may be z 's immediate right child (case c) or deeper (case d)
- TRANSPLANT does not update children of v : caller's responsibility (TREE-DELETE handles this)
- Successor when right subtree empty: must go up tree until finding a node that is a left child
- Successor has no left child: by property of inorder successor (Exercise 12.2-5)
- BST sort worst-case: inserting sorted keys $\Theta(n)$ — each insert at end of linked-list tree

Diagrams & Visuals

Binary search tree (h=2):



Inorder: 2,5,5,6,7,8

Search path for k=13:

15 → 6 → 7 → 13
(go left at 15, right at 6, right at 7, found)

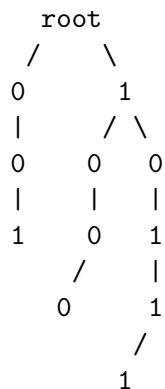
Successor of node 13 (no right child):

13 → parent 7 (13 is right child) → parent 6 (7 is right child) → parent 15 (6 is left child)

Delete cases:

- (a) z no left child: z.right replaces z
- (b) z no right child: z.left replaces z
- (c) z two children, y=successor is z.right: y replaces z
- (d) z two children, y deeper: splice y out first, then y replaces z

Radix tree (trie) for strings {0,011,10,100,1011}:



End-of-Chapter Material

- Key terms: binary search tree, BST property, inorder/preorder/postorder tree walk, successor, predecessor, trailing pointer, TRANSPLANT, radix tree (trie), Catalan number, randomly built BST, total path length
- Theorem 12.1: inorder walk $\Theta(n)$
- Theorem 12.2: SEARCH, MIN/MAX, SUCC/PRED $O(h)$
- Theorem 12.3: INSERT, DELETE $O(h)$
- Exercises 12.1: draw BSTs of heights 2-6; BST vs min-heap property; nonrecursive inorder walk (stack or threaded)
- Exercises 12.2: search path validity; recursive MIN/MAX; successor has no left

child; $\Theta(n)$ inorder via MINIMUM + $n-1$ SUCCESSOR calls; k successive SUCCESSOR calls take $O(k+h)$

- Exercises 12.3: BST sort (worst $\Theta(n)$, best $\Theta(n \lg n)$); deletion not commutative; successor-only representation; predecessor as alternative for deletion
- Problems 12-1: BST with equal keys — strategies: boolean flag (alternate), list at node, random; analysis of insertion time
- Problems 12-2: Radix trees (trie) — lexicographic sort of bit strings in $\Theta(\text{total length})$ time
- Problems 12-3: Average node depth in randomly built BST is $O(\lg n)$; expected total path length $O(n \lg n)$; links to quicksort analysis
- Problems 12-4: Catalan numbers — $b = 1$; recurrence $\sum b_{n-1-k}$; number of distinct binary trees

Cross-Chapter Links

- Requires: Ch. 10 (binary tree representation, linked lists, left-child right-sibling), Appendix B (tree math), Ch. 2 (sorting lower bound)
 - Referenced in: Ch. 13 (red-black trees — inherits all BST operations), Ch. 14 (optimal BST), Ch. 17 (order-statistic trees, interval trees), Ch. 18 (B-trees), Ch. 19 (disjoint-set forests)
-

Ch. 13 — Red-Black Trees

Named Entities (Terms & Definitions)

- Red-black tree: BST with 1 extra bit per node (color: RED or BLACK); approximately balanced; height $\leq 2 \lg(n+1)$
- Red-black properties: (1) every node red or black (2) root is black (3) every leaf (NIL sentinel) is black (4) red node $\rightarrow$ both children black (5) all paths from node to descendant leaves have same #black nodes
- Black-height $bh(x)$: number of black nodes on path from x (excluding x) down to a leaf
- Sentinel $T.nil$: single object representing all NIL leaves; color BLACK; replaces all NIL pointers; occupies $O(1)$ space
- Rotation: local pointer rearrangement preserving BST property; left rotation and right rotation; $O(1)$ time
- Left rotation: $x.right = y$ becomes new root of subtree; x becomes y 's left child; y 's left child (β) becomes x 's right child
- Right rotation: symmetric; $y.left = x$ becomes root; y becomes x 's right child
- Uncle: sibling of a node's parent
- Doubly black: conceptual extra black on node x after deletion of black y ; x contributes 2 black count on paths
- Red-and-black: conceptual state when x is RED after deletion; contributes 1 black; colored red by color attribute
- AVL tree: height-balanced BST; $|\text{left.h} - \text{right.h}| \leq 1$ for all nodes; height $h \leq 1.44 \lg n$; $O(\lg n)$ rotations per insert

- Persistent data structure: maintains past versions; copying only path nodes ($O(h)$ per op)
- Join operation (RB-JOIN): merge two red-black trees S_1, S_2 and element x where all keys in $S_1 \leq x.key \leq$ all keys in S_2 ; $O(\log n)$
- Left-leaning red-black tree: variant by Sedgwick where red nodes are only left children; simpler code but not constant rotations

Processes / Algorithms / Pathways

LEFT-ROTATE(T, x)

- Goal: Perform left rotation on node x ($x.right \neq T.nil$)
- Steps: (1) $y = x.right$ (2) $x.right = y.left$ (3) if $y.left \neq T.nil$: $y.left.p = x$ (4) $y.p = x.p$ (5) if $x.p == T.nil$: $T.root = y$ (6) elseif $x == x.p.left$: $x.p.left = y$ (7) else: $x.p.right = y$ (8) $y.left = x$ (9) $x.p = y$
- Complexity: $O(1)$; preserves BST order: $\alpha < x < \beta < y < \gamma$

RIGHT-ROTATE(T, y)

- Complexity: $O(1)$; symmetric

RB-INSERT(T, z)

- Goal: Insert node z into red-black tree maintaining red-black properties
- Steps: (1) Same as TREE-INSERT but with $T.nil$ sentinel (2) set $z.left = z.right = T.nil$ (3) $z.color = RED$ (4) call RB-INSERT-FIXUP(T, z)
- Four differences from TREE-INSERT: (a) $NIL \rightarrow T.nil$ (b) set $z.left = z.right = T.nil$ (c) color z RED (d) call FIXUP
- Complexity: $O(\log n)$; at most 2 rotations

RB-INSERT-FIXUP(T, z)

- Goal: Restore red-black properties after insertion
- Loop invariant: (a) z is red (b) if $z.p$ is root, $z.p$ is black (c) at most one violation (property 2 root-black or property 4 no-red-red)
- Symmetric cases: lines 3-15 for $z.p$ left child; lines 17-29 for $z.p$ right child
- Case 1 (uncle y red): recolor: $z.p=BLACK, y=BLACK, z.p.p=RED$; z moves up two levels; only case that repeats the loop
- Case 2 (uncle black, z right child): left rotation on $z.p$; falls through to Case 3
- Case 3 (uncle black, z left child): recolor $z.p=BLACK, z.p.p=RED$; right rotation on $z.p.p$; terminates
- Termination: when $z.p$ is black; line 30 colors root black
- Complexity: $O(\log n)$; ≤ 2 rotations (only Case 1 repeats; Cases 2+3 terminate)
- Example: Insert 38,31,12. 38(root,black). 31(left,red). 12(left of 31,red) $\rightarrow$ Case 1: uncle= NIL (black) $\rightarrow$ Case 3: recolor 31= $BLACK, 38=RED$, right-rotate on 38 $\rightarrow$ 31(black), left=12(red), right=38(red)

RB-TRANSPLANT(T, u, v)

- Steps: (1) if $u.p == T.nil$: $T.root = v$ (2) elseif $u == u.p.left$: $u.p.left = v$ (3) else: $u.p.right = v$ (4) $v.p = u.p$ (unconditional — differs from TRANSPLANT)
- Difference from TRANSPLANT: uses $T.nil$; unconditional $v.p$ assignment

RB-DELETE(T, z)

- Goal: Delete node z from red-black tree
- Steps: (1) $y = z$; $y\text{-original-color} = y.color$ (2) if z has ≤ 1 child: set $x = \text{child}$, RB-TRANSPLANT(z , child) (3) else: $y = \text{TREE-MINIMUM}(z.right)$; $y\text{-original-color} = y.color$; $x = y.right$; splice y out if not $z.right$; RB-TRANSPLANT(z , y); $y.color = z.color$ (4) if $y\text{-original-color} == \text{BLACK}$: call RB-DELETE-FIXUP(T , x)
- Key tracking: y = node removed/moved; x = node taking y 's place (may be $T.nil$); $y\text{-original-color}$ determines if fixup needed
- Complexity: $O(\log n)$; at most 3 rotations
- If y red: no violation — black-heights unchanged, no adjacent reds, root unchanged

RB-DELETE-FIXUP(T, x)

- Goal: Fix red-black properties after deletion of a black node
- Idea: x is “doubly black” (if $x.color = \text{BLACK}$) or “red-and-black” (if $x.color = \text{RED}$); push extra black up tree until it vanishes
- Goal of while loop: move extra black up until (1) x is red-and-black $\rightarrow$ color black (2) x is root $\rightarrow$ extra black vanishes (3) rotations + recolorings fix it
- Symmetric cases: lines 3-22 for x left child; lines 24-43 for x right child
- Case 1 (sibling w red): $w = \text{BLACK}$, $x.p = \text{RED}$, left-rotate on $x.p$; new $w = x.p.right$ (black); falls into 2/3/4
- Case 2 (w black, both w 's children black): remove one black from x and w ($w = \text{RED}$); move extra black up to $x.p$; $x = x.p$; only case that repeats
- Case 3 (w black, $w.left$ red, $w.right$ black): $w.left = \text{BLACK}$, $w = \text{RED}$, right-rotate on w ; new $w = x.p.right$ (black with red right child); falls into Case 4
- Case 4 (w black, $w.right$ red): $w.color = x.p.color$; $x.p = \text{BLACK}$; $w.right = \text{BLACK}$; left-rotate on $x.p$; $x = T.root$ (terminates)
- Termination: Cases 1,3,4 each terminate; only Case 2 repeats (at most $O(\log n)$ times)
- Complexity: $O(\log n)$; at most 3 rotations

Classifications & Hierarchies

- Balanced BSTs: red-black (Ch. 13), AVL (Problem 13-3), AA-trees (never red left child), treaps (BST+heap), splay trees (self-adjusting), scapegoat trees, weight-balanced trees
- Red-black variants: left-leaning red-black trees (Sedgewick); all red nodes are left children; simpler but non-constant rotations

Comparisons & Trade-offs

Dimension	Red-Black Tree	AVL Tree
Height bound	$\leq 2 \lg(n+1)$	$\leq 1.44 \lg n$ (stricter)
Insert rotations	≤ 2	$O(\log n)$
Delete rotations	≤ 3	$O(\log n)$
Lookup speed	$O(\log n)$	Slightly faster (tighter balance)
Implementation	More complex	Simpler (no color; just height)

Dimension	BST (Ch. 12)	Red-Black Tree
Worst-case height	$\Theta(n)$ (degenerate)	$O(\log n)$
SEARCH, INSERT, DELETE	$O(h)$	$O(\log n)$
Space	n nodes + 3 pointers	n nodes + 3 ptrs + 1 color bit
Sentinel	Optional	Required (T.nil)

Formulas & Equations

Height bound (Lemma 13.1) $h \leq 2 \lg(n+1)$ - Proof: (1) subtree at x contains $\geq 2^{bh(x)} - 1$ internal nodes (induction on height) (2) root black-height $\geq h/2$ (property 4: at least half nodes on any path are black) (3) $n \geq 2^{(h/2)} - 1 \rightarrow h \leq 2 \lg(n+1)$

Black-height bounds $h/2 \leq bh(\text{root}) \leq h$

Rules, Laws & Theorems

Lemma 13.1 (Height bound)

- Red-black tree with n internal nodes has height $\leq 2 \lg(n+1)$
- Implies $O(\log n)$ for all BST operations

Lemma 13.1 proof detail (induction on height)

- Base: height 0 node (T.nil) has $2^0 - 1 = 0$ internal nodes
- Inductive: each child of x has bh of either $bh(x)-1$ (if black) or $bh(x)$ (if red)
- Each child has $\geq 2^{(bh(x)-1)} - 1$ internal nodes; subtree has $\geq 2(2^{(bh(x)-1)} - 1) + 1 = 2^{bh(x)} - 1$

Property 4 consequence (Exercise 13.1-8)

- A red node cannot have exactly one non-NIL child
- Proof: if red node had one child, that child's black-height would differ from NIL leaf's

RB-INSERT-FIXUP invariant at loop test

- (a) z is red; (b) if $z.p$ is root, it is black; (c) at most one violation (property 2 or 4)

RB-DELETE-FIXUP

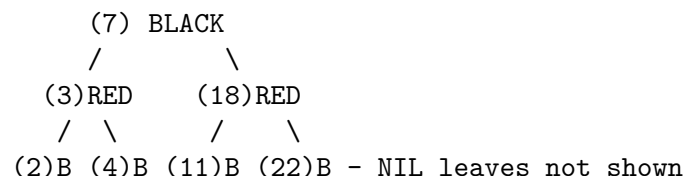
- Only Case 2 repeats the loop; Cases 1,3,4 terminate; at most $O(\log n)$ iterations total

Edge Cases & Common Pitfalls

- Inserting black instead of red: would violate property 5 (extra black on paths through new node)
- Deleting red node: no problem — properties preserved (Exercise 13.4-1)
- Deleting black root with red child: new root (x) is red $\rightarrow$ property 2 violated; fix by coloring root black (line 44)
- T.nil sentinel: must never be colored RED; RB-INSERT-FIXUP never sets T.nil.color = RED (Exercise 13.3-4)
- Case 2 entering from Case 1: new x (x.p) will be red-and-black $\rightarrow$ loop terminates next iteration by coloring black
- Doubly black: not a real color attribute; conceptual; x.color attribute stays RED or BLACK
- x.p must be set in RB-DELETE: even when $x = T.nil$ (line 16), needed by RB-DELETE-FIXUP
- RB-TRANSPLANT unconditional v.p: differs from TRANSPLANT; needed because FIXUP reads x.p even if x is T.nil
- Worst-case ratio red:black (Exercise 13.1-7): complete binary tree with alternating red/black = red count = black count (ratio 1); all black = ratio 0

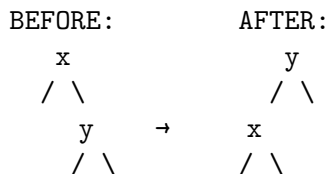
Diagrams & Visuals

Red-black tree structure:



Black-heights: nil=0, 2=bh1, 4=bh1, 3=bh1, 11=bh1, 22=bh1, 18=bh1, 7=bh2

Left rotation:



BST order preserved: $x < y$

RB-INSERT-FIXUP cases (z.p left child):

Case 1: z.p and uncle both red $\rightarrow$ recolor, move z up 2

Case 2: z.p red, uncle black, z right child $\rightarrow$ left rotate $\rightarrow$ Case 3

Case 3: z.p red, uncle black, z left child → recolor + right rotate → done

RB-DELETE-FIXUP cases (x left child):

Case 1: w red → recolor + left rotate → Case 2/3/4

Case 2: w black, both children black → push black up (only repeat case)

Case 3: w black, w.left red, w.right black → right rotate → Case 4

Case 4: w black, w.right red → recolor + left rotate → done

End-of-Chapter Material

- Key terms: red-black tree, red-black properties, black-height, sentinel T.nil, rotation (left/right), RB-INSERT, RB-INSERT-FIXUP, RB-DELETE, RB-DELETE-FIXUP, doubly black, red-and-black, uncle, AVL tree, persistent set, join, left-leaning red-black tree
- Lemma 13.1: $h \leq 2 \lg(n+1)$
- Exercises 13.1: draw RB tree; red node cannot have exactly one child; largest/smallest internal nodes for given black-height; ratio of red to black
- Exercises 13.2: any BST can be transformed to any other using $O(n)$ rotations; right-conversion $O(n)$; exactly $n-1$ possible rotations in n -node BST
- Exercises 13.3: trace insertions 41,38,31,12,19,8 into empty RB tree; black-height labeling; T.nil never set RED
- Exercises 13.4: trace deletions; sentinel in FIXUP; insert-then-delete may not restore original tree
- Problems 13-1: Persistent BST — insert/deletion copies $O(h)$ nodes per operation; parent pointers make it $\Omega(n)$; RB tree gives $O(\log n)$ per op
- Problems 13-2: RB-JOIN — graft two red-black trees separated by element x; find node of matching black-height; $O(\log n)$ time
- Problems 13-3: AVL trees — height $O(\log n)$, at least F_h nodes (Fibonacci); BALANCE procedure using rotations; AVL-INSERT $O(\log n)$ with $O(\log n)$ rotations

Cross-Chapter Links

- Requires: Ch. 12 (BST operations — TREE-INSERT, TREE-DELETE, TREE-MINIMUM, SUCCESSOR), Ch. 10 (binary tree representation, sentinel)
- Referenced in: Ch. 14 (optimal BST comparison), Ch. 16 (amortized analysis — constant rotations per op matters for potential method), Ch. 17 (order-statistic trees and interval trees use RB trees as base), Ch. 18 (B-trees compared), Ch. 19 (mergeable heaps)